

Table of Contents

1 Introduction

- 1.1 Aims and Objectives
- 1.2 Background Information
- 1.3 Description of the Flight Dataset

2 Analytics Design

3 Data Analytics

- 3.1 Data Pre-processing
 - 3.1.1 Data Cleaning
 - 3.1.2 Data Transformation
 - 3.1.3 Data Sub setting
 - 3.1.4 Exploratory Data Analysis
 - 3.1.5 Data Visualization
- 3.2 Machine Learning
 - 3.2.1 Linear regression modelling
 - 3.2.2 Model evaluation.

4 Critical Comparison of SAS and Python

- 4.1 Introduction
- 4.2 SAS University Edition Overview
- 4.3 Python Overview
- 4.4 Evaluation of SAS University Edition and Python (Tool's efficiency, User-friendliness, Scalability for large datasets, ease of use, Data visualization capabilities).

5 Conclusion

- 5.1 Analytics Insights
- 5.2 Limitations and Future Works

6 References

7 Appendix 1 – SAS Code

8 Appendix 2 – Python Code

1 Introduction

1.1 Aims and Objectives

The aim of this study is to analyze the flight booking dataset gotten from “Ease My Trip” website to gain insights into the factors influencing flight ticket prices and to develop a predictive model using linear regression to accurately estimate ticket prices based on key features such as airline, class, number of stops, and duration. By understanding the relationships between these variables and ticket prices, we aim to provide valuable information for both consumers and airlines, ultimately improving decision-making processes and optimizing pricing strategies within the aviation industry.

The objective of the study is to:

- Explore Factors Influencing Ticket Prices: Identify and analyze the key factors that influence flight ticket prices, including airline, class, number of stops, departure/arrival times, and duration.
- Visualize Price Variations: Visualize the variations in ticket prices based on different categorical variables such as airline, class, departure/arrival times, and number of stops to understand the patterns and trends.
- Evaluate Impact of Time: Evaluate the impact of time-related factors such as days left before departure, departure/arrival times on ticket prices, and analyze how these variables affect pricing strategies.
- Develop Predictive Model: Develop a predictive model using linear regression to accurately estimate flight ticket prices based on selected features and evaluate its performance using appropriate metrics such as mean squared error, mean absolute error, and R-squared.
- Provide Insights and Recommendations: Provide actionable insights and recommendations to stakeholders in the aviation industry, including airlines and consumers, based on the analysis results and predictive model. These insights can inform pricing strategies, marketing efforts, and consumer decision-making processes to enhance competitiveness and profitability.

The study will attempt to answer the following business questions:

- Does price vary with Airlines?
- How is the price affected when tickets are bought in just 1 or 2 days before departure?
- How does the price change with change in Source and Destination?
- How does the ticket price vary between Economy and Business class?

1.2 Background Information

“Ease My Trip”

Founded in 2008, EaseMyTrip is a prominent travel platform in India, recognized as a Unicorn-listed company. Offering a comprehensive range of travel services, including flights, hotels, holidays, cabs, and buses, they cater to the needs of over 20 million users with a customer-

centric approach. Through dedicated professionals and advanced technology, they've expanded their presence across PAN India and international destinations. Their commitment to excellence extends to corporate travel solutions, providing entrepreneurs with streamlined journeys, round-the-clock assistance, and cost-efficient operations. ([About Us and Company Profile - EaseMyTrip.com](#))

1.3 Description of the Flight Dataset

The dataset used for this study contains information about flight booking options from the Easemytrip website for flight travel between India's top 6 metro cities. There are 300261 datapoints and 11 features in the cleaned dataset. This dataset was obtained from Kaggle website.

Feature	Description	Data Type
Airline	Name of the airline company	Categorical
Flight	Flight code	Categorical
Source City	City from which the flight takes off	Categorical
Departure Time	Time periods grouped into bins representing departure time	Categorical
Stops	Number of stops between source and destination cities	Categorical
Arrival Time	Time intervals grouped into bins representing arrival time	Categorical
Destination City	City where the flight will land	Categorical
Class	Seat class (Business or Economy)	Categorical
Duration	Overall amount of time it takes to travel between cities (hours)	Continuous
Days Left	Difference between trip date and booking date	Continuous
Price	Ticket price	Continuous

Table 1 – Description of key features in the Flight dataset.

2 Analytics Design

- **Data Collection and Integration:** We obtained the flight booking dataset from Kaggle which was originally gotten from Ease My Trip website using web scraping techniques. The data collected included information such as airline, flight code, source city, departure time, stops, arrival time, destination city, class, duration, days left, and price. The collected data was integrated into a structured format suitable for analysis.
- **Data Preprocessing:**
 - Data Cleaning:** We performed data cleaning to handle missing values and ensure data integrity.
 - Data Transformation:** Certain variables were transformed, such as converting flight duration from hours to minutes and converting prices from Indian rupees to US dollars.

Data Subsetting: We focused our analysis on specific features relevant to our research questions.

- **Data Analytics:**
Exploratory Data Analysis (EDA): We conducted EDA to gain insights into the relationships between different variables in the dataset. Visualization techniques such as bar plots, line plots, box plots, and KDE plots were used to represent data distributions and identify patterns.
- **Machine Learning:**
Linear Regression Modelling: We employed linear regression to predict flight prices based on features such as airline, class, stops, and duration.
Model Evaluation: Performance metrics such as mean squared error (MSE), mean absolute error (MAE), and R-squared were used to assess the accuracy of the predictive model.
- **Reporting and Visualization:** The findings of our analysis were presented in a comprehensive report to stakeholders. Visualizations such as bar plots, line plots, box plots, and KDE plots were included in the report to illustrate key insights and trends.

Flight Data Analytics – UML Activity Diagram

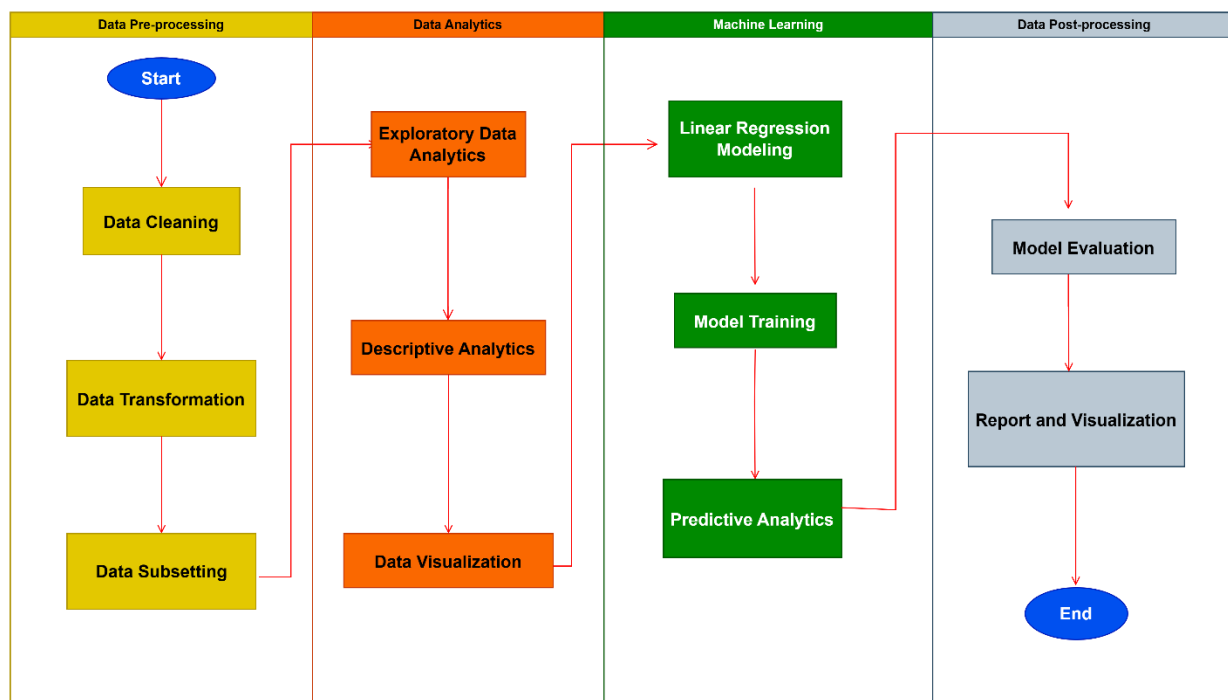


Figure 1 – UML Activity Diagram for Flight Analytics Design.

3 Data Analytics

The analysis carried out in this section was completed using both Python and SAS university edition. The code used to produce the results displayed can be seen in the Appendix section of this report.

3.1 Data Pre-processing

At the stage of data pre-processing a range of techniques were applied to the original flight dataset in preparation for the machine learning activities.

3.1.1 Data Cleaning

We began by cleaning the dataset to handle missing values and ensure data integrity. No significant data cleaning was required as the dataset was relatively clean.

3.1.2 Data Transformation

We transformed certain variables to make them more suitable for analysis. Notably, we converted the duration of flights from hours to minutes and converted prices from Indian rupees to US dollars.

Obs	VARI	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class	duration_hour	days_left	Price_Indian_Rupee	duration_minutes	Price_US_dollars
1	0	SpiceJet	SG-8709	Delhi	Evening	zero	Night	Mumbai	Economy	2.17	1	5953	130.2	\$71.44
2	1	SpiceJet	SG-8157	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.33	1	5953	139.8	\$71.44
3	2	AirAsia	I5-764	Delhi	Early_Morning	zero	Early_Morning	Mumbai	Economy	2.17	1	5956	130.2	\$71.47
4	3	Vistara	UK-995	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.25	1	5955	135.0	\$71.46
5	4	Vistara	UK-963	Delhi	Morning	zero	Morning	Mumbai	Economy	2.33	1	5955	139.8	\$71.46
6	5	Vistara	UK-945	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.33	1	5955	139.8	\$71.46
7	6	Vistara	UK-927	Delhi	Morning	zero	Morning	Mumbai	Economy	2.08	1	6060	124.8	\$72.72
8	7	Vistara	UK-951	Delhi	Afternoon	zero	Evening	Mumbai	Economy	2.17	1	6060	130.2	\$72.72
9	8	GO_FIRST	G8-334	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.17	1	5954	130.2	\$71.45
10	9	GO_FIRST	G8-336	Delhi	Afternoon	zero	Evening	Mumbai	Economy	2.25	1	5954	135.0	\$71.45
11	10	GO_FIRST	G8-392	Delhi	Afternoon	zero	Evening	Mumbai	Economy	2.25	1	5954	135.0	\$71.45
12	11	GO_FIRST	G8-338	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.33	1	5954	139.8	\$71.45
13	12	Indigo	6E-5001	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.17	1	5955	130.2	\$71.46
14	13	Indigo	6E-6202	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.17	1	5955	130.2	\$71.46
15	14	Indigo	6E-549	Delhi	Afternoon	zero	Evening	Mumbai	Economy	2.25	1	5955	135.0	\$71.46
16	15	Indigo	6E-6278	Delhi	Morning	zero	Morning	Mumbai	Economy	2.33	1	5955	139.8	\$71.46
17	16	Air_India	AI-887	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.08	1	5955	124.8	\$71.46
18	17	Air_India	AI-665	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.17	1	5955	130.2	\$71.46
19	18	AirAsia	I5-747	Delhi	Evening	one	Early_Morning	Mumbai	Economy	12.25	1	5949	735.0	\$71.39
20	19	AirAsia	I5-747	Delhi	Evening	one	Morning	Mumbai	Economy	16.33	1	5949	979.8	\$71.39

Table 2 – conversion of duration of flights from hours to minutes and prices from Indian rupees to US dollars.

3.1.3 Data Sub-setting

We focused our analysis on specific features relevant to our research questions, such as airline, departure time, arrival time, source city, destination city, class, duration, days left, stops, and price.

3.1.4 Exploratory Data Analysis (EDA):

Through EDA, we gained insights into the relationships between different variables in the dataset. This included analyzing the distribution of flight prices across various factors such as airlines, departure time, arrival time, source city, destination city, class, and number of stops.

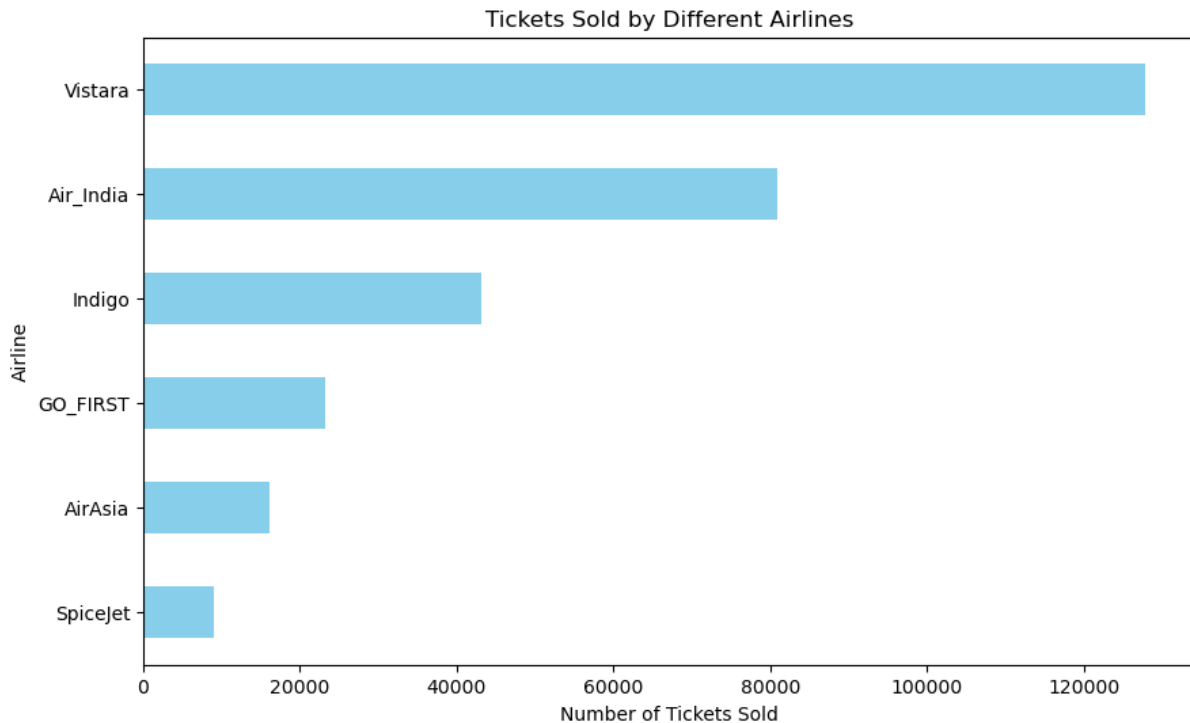


Figure 2 – Bar plot of tickets sold by different airlines.

Figure 2 bar chart illustrates the number of tickets sold by different airlines on the Ease My Trip website. The chart provides insights into which airlines are the most popular among travelers booking flights through Ease My Trip. Vistara airline emerges as the top performer, with the highest number of tickets sold, followed by Air_India and Indigo. By analyzing the proportion of tickets sold by each airline, we can assess their market share and competitive position within the industry. Understanding the popularity of each airline can inform strategic decisions related to partnerships, marketing campaigns, and route optimization.

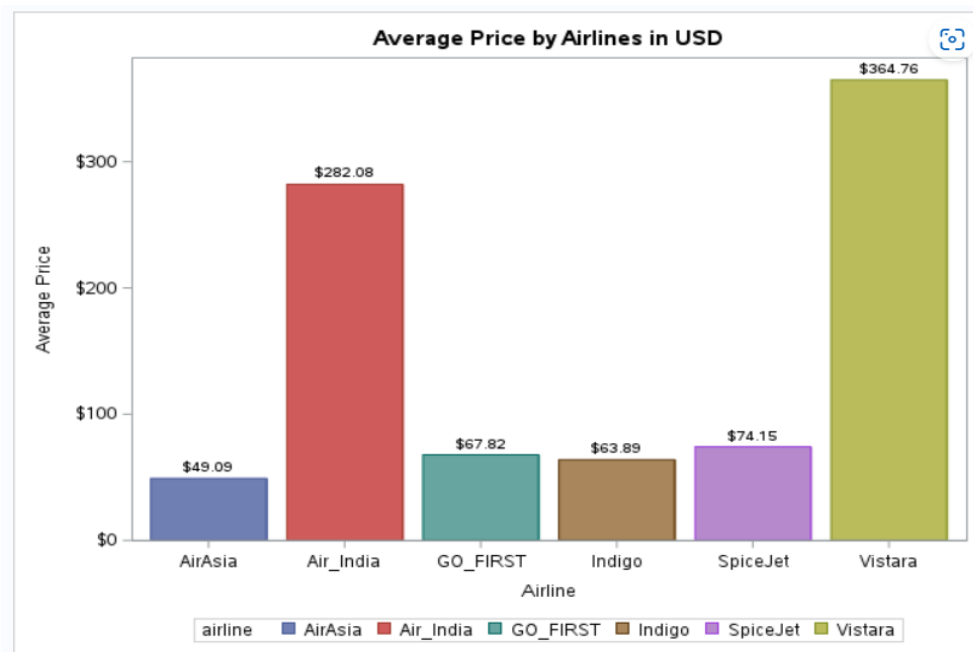


Figure 3 – Bar plot showing the Average Price by Airlines in USD

Figure 3 bar plot illustrates the average price of flight tickets for each airline represented in the dataset. There is noticeable variation in the average price of tickets across the different airlines. Vistara airline have higher average ticket prices followed by Air_India compared to the other airlines, indicating potential differences in pricing strategies or service offerings.

This analysis provides stakeholders with insights into the pricing dynamics of various airlines operating in the market. Understanding the pricing landscape can help stakeholders make informed decisions regarding partnerships, pricing adjustments, and competitive positioning.

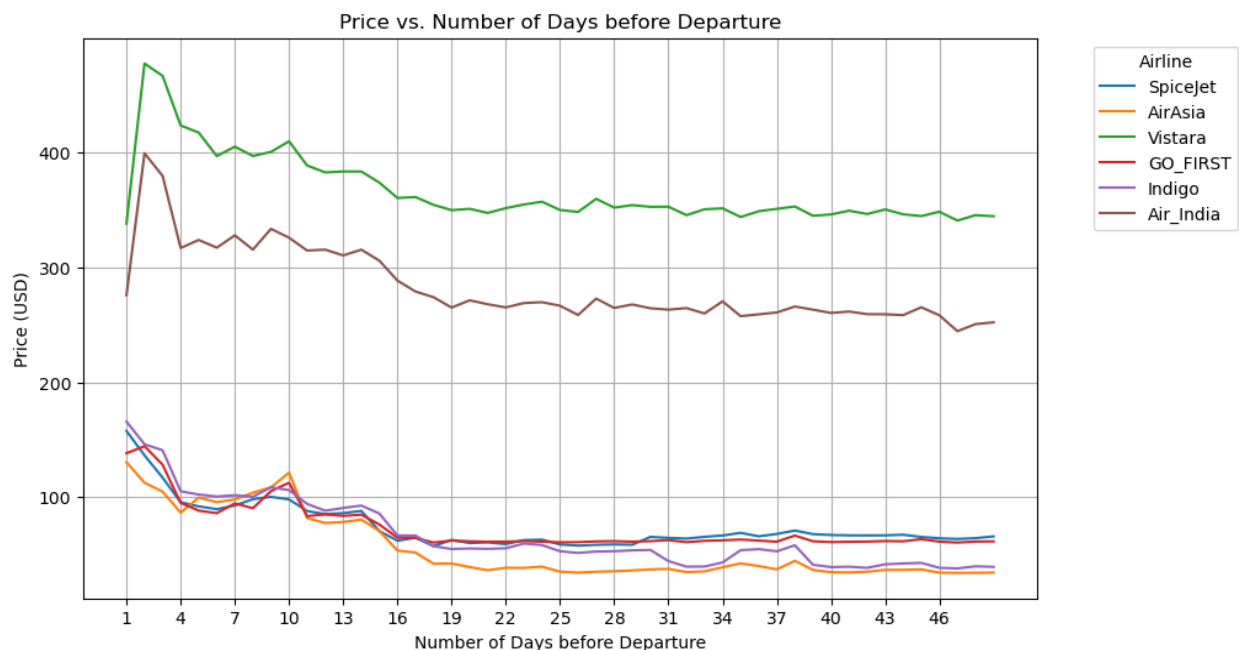


Figure 4 – Line plot showing Price vs. Number of Days before Departure

Figure 4-line plot illustrates how ticket prices vary based on the number of days remaining before departure. Prices tend to increase significantly when tickets are purchased just one or two days before departure, irrespective of the airline. While most airlines show a similar trend of price escalation close to the departure date, Vistara and Air India stand out for offering lower prices on the last day compared to two days before departure.

The analysis highlights the impact of booking timing on flight ticket prices, with prices generally increasing significantly when tickets are purchased just one or two days before departure. By leveraging these insights, stakeholders can develop proactive pricing strategies and promotional campaigns to optimize revenue and enhance customer satisfaction.

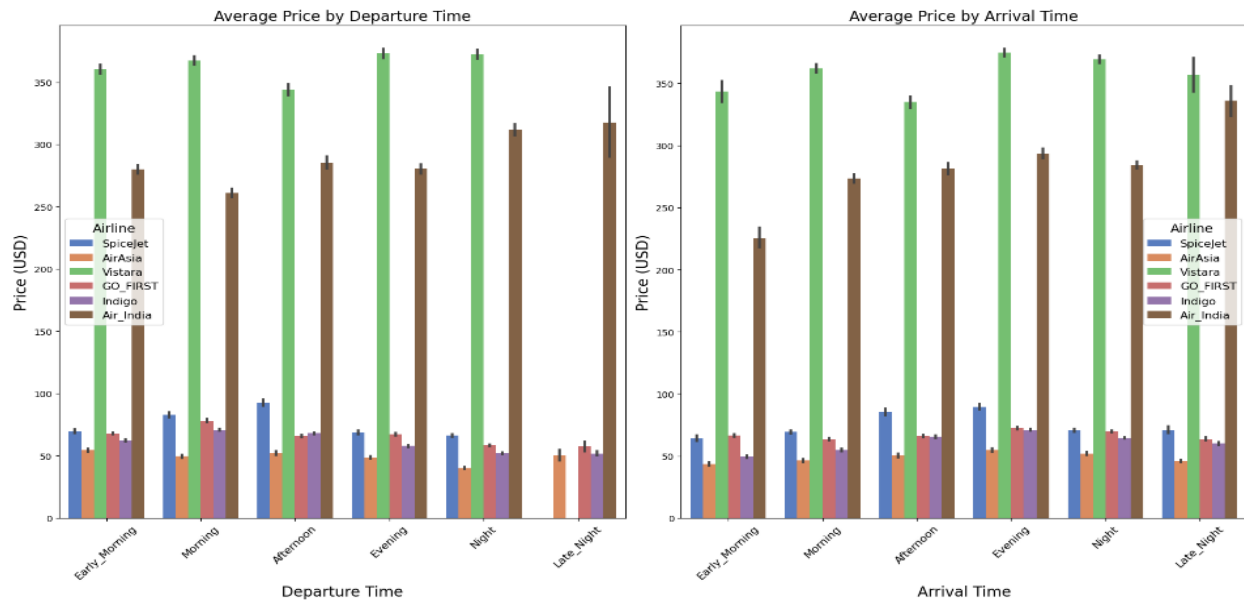


Figure 5: Bar plots showing Average Price by Departure time and Arrival Time

From figure 5, the bar plots revealed fluctuations in average ticket prices based on departure and arrival times. For departure, SpiceJet average price is highest in the afternoon, Indigo average price is highest in the morning, GO_FIRST average price is highest in the morning, AirAsia average price is highest at late-night, Air India average price is highest at late-night, Vistara average price is highest in the evening and night. While for arrival, Vistara, AirAsia, Indigo, Spicejet, GO_FIRST, have their highest average price in the evening, except for Air India which have its highest average price at late-night.

This analysis provides valuable insights into the impact of departure and arrival times on ticket prices, allowing stakeholders to optimize pricing strategies and revenue management practices. By understanding the pricing patterns associated with different time periods, stakeholders can tailor pricing strategies to capitalize on periods of high demand or adjust prices to attract customers during off-peak times.

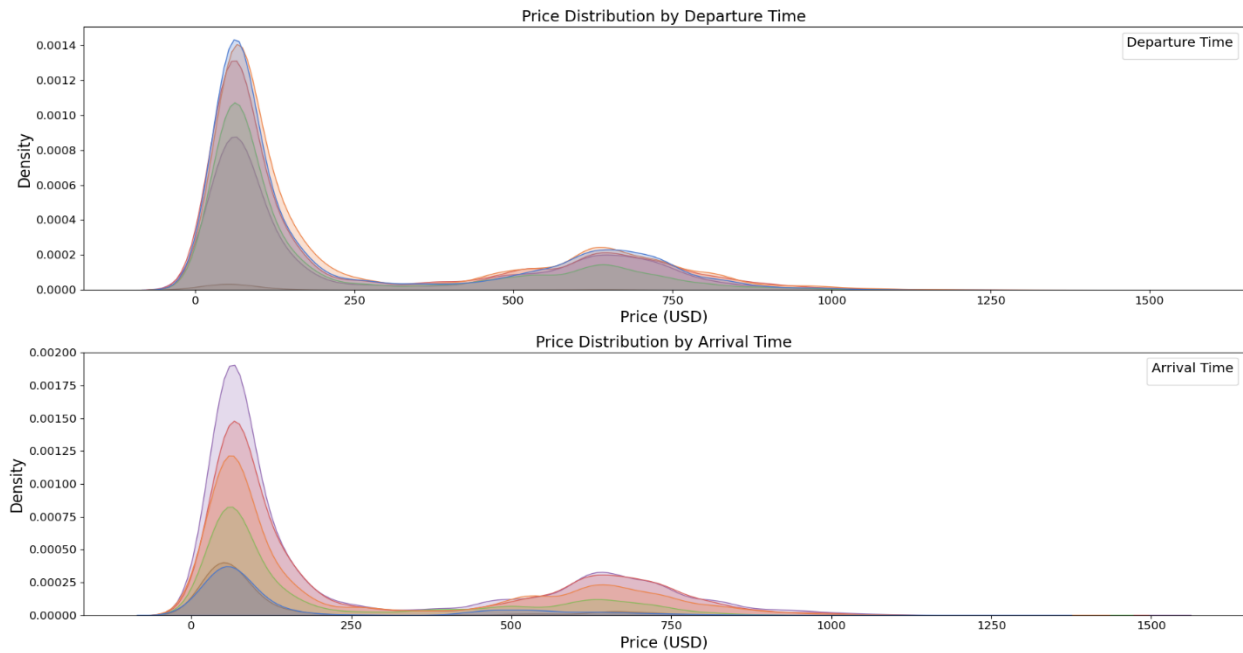


Figure 6 – Kernel Density Estimation (KDE) Plots showing price distribution by departure and arrival time.

Flights departing or arriving late at night tend to be cheaper than those at other times. The price distributions for different departure and arrival times are significantly different from a normal distribution. This suggests that extreme values might be influencing our analysis. However, focusing on the median values, which are less affected by extreme values, we find that our previous conclusion still holds. Flights departing or arriving late at night remain cheaper than others. Interestingly, the median prices in other time groups are similar, indicating that late-night flights consistently offer better deals.

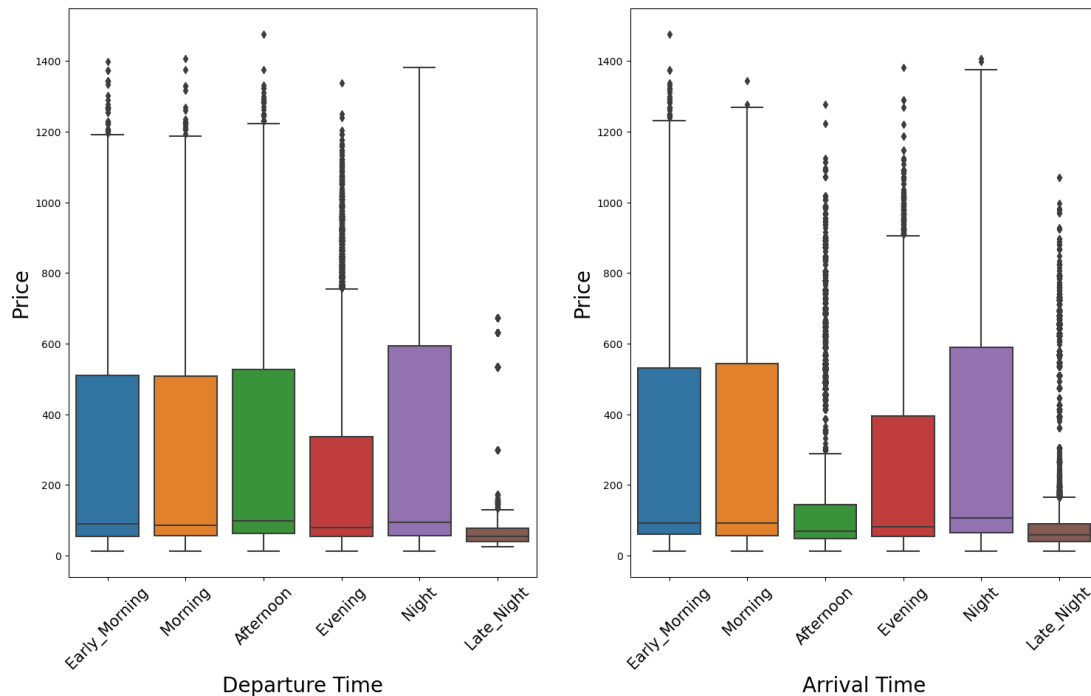


Figure 7 – Box plot showing price for departure and arrival time.

Figure 7 reinforces our earlier observation that flights departing or arriving late at night tend to be cheaper than others. Despite the significant differences in price distribution from a normal distribution, focusing on median values, which are less affected by extreme values, we find that our previous conclusion still holds. Flights departing or arriving late at night remain cheaper than others. Interestingly, the median prices in other time groups are similar, indicating that late-night flights consistently offer better deals.

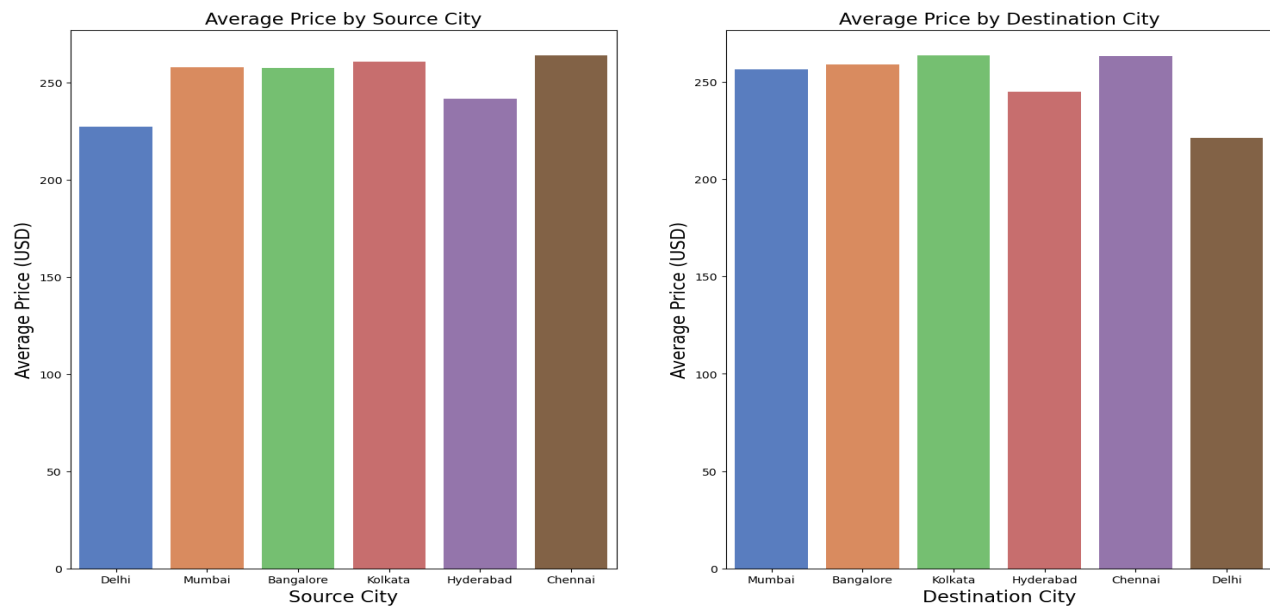


Figure 8 – Box plot showing Average Price by Source and Destination City



Figure 9 – Line plot showing Price Distribution by Source City and Destination City

Figure 8 illustrates the variations in average ticket prices depending on the source and destination cities. Delhi has the lowest average price for source city while Kolkata and Chennai have the highest average price for destination cities, indicating differences in pricing dynamics based on geographical factors.

Figure 9 shows the distribution of ticket prices based on the source and destination cities. The distributions indicate that ticket prices vary significantly based on both source and destination cities. The differences in price distribution could be attributed to factors such as distance, popularity of routes, and demand-supply dynamics.

The analysis highlights the significant impact of source and destination cities on ticket prices, with variations observed across different geographical regions. By leveraging these insights, stakeholders can develop data-driven pricing strategies to maximize revenue and enhance customer satisfaction in specific market segments.

3.1.5 Data Visualization

Visualization techniques such as bar plots, line plots, box plots, and KDE plots were used to represent data distributions and identify patterns.

The bar plot of figure 10 shows the comparison of ticket prices between Economy and Business class revealed significant differences in average prices. Business class tickets exhibited notably higher average prices compared to Economy class, indicating premium pricing for enhanced services and amenities.

The pie chart depicted the distribution of tickets between Economy and Business class, showcasing the proportion of passengers opting for each class. While Economy class accounted for most ticket sales, Business class represented a smaller yet significant portion of the market.

Understanding the price differentials between Economy and Business class is crucial for revenue management and pricing strategies. Stakeholders can leverage these insights to optimize pricing structures and maximize revenue potential across different passenger segments.

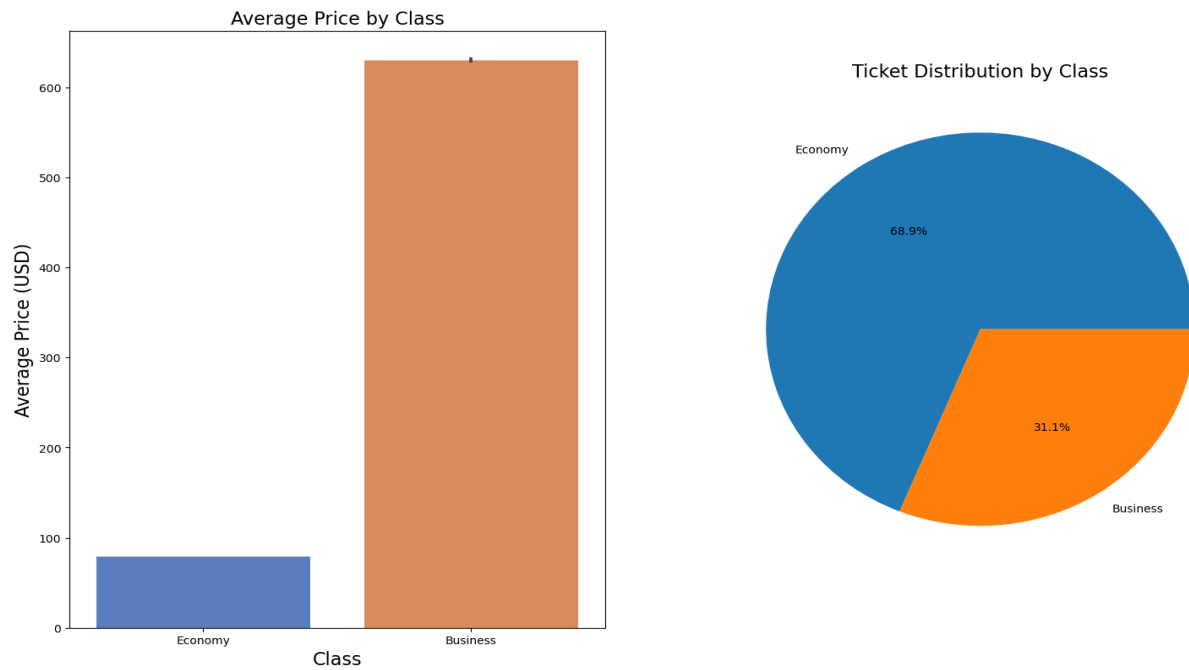


Figure 10 – Bar plot showing Average Price by Class and Pie chart showing Ticket Distribution by Class.

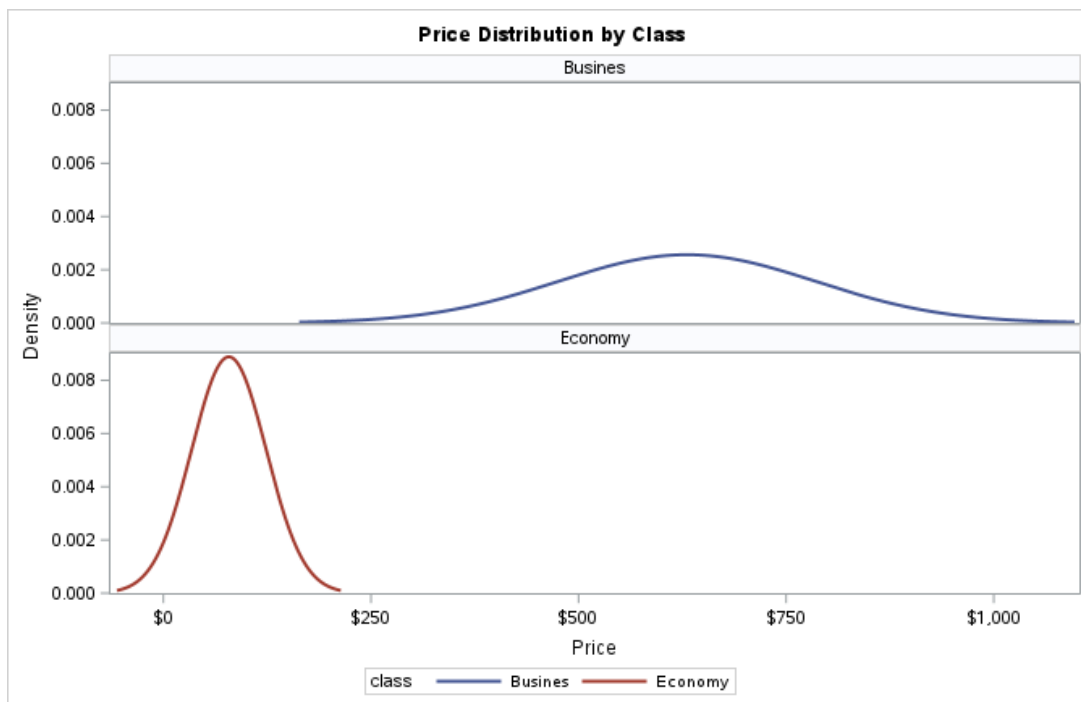


Figure 11 – Line plot visualizing price distribution by class.

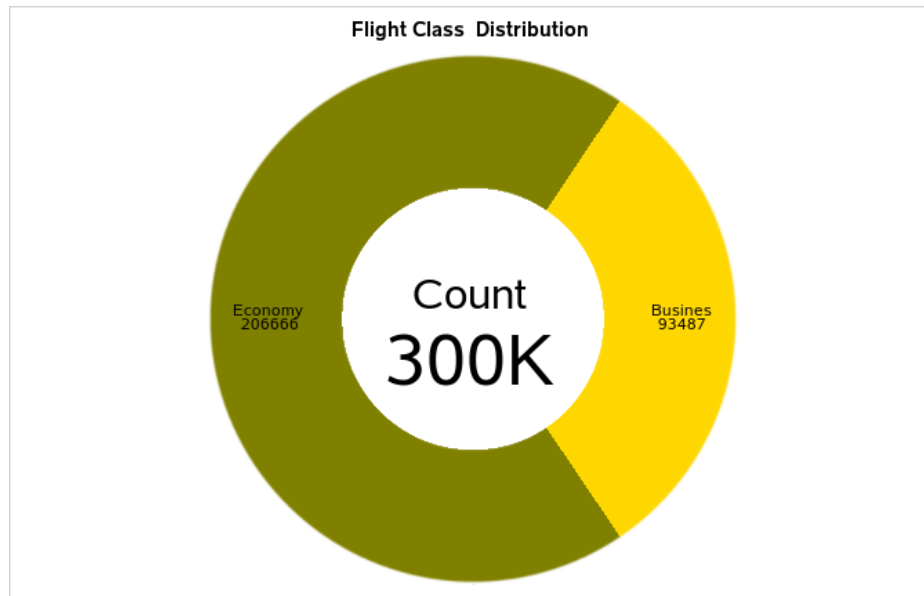


Figure 12 – Doughnut plot visualizing flight class distribution.

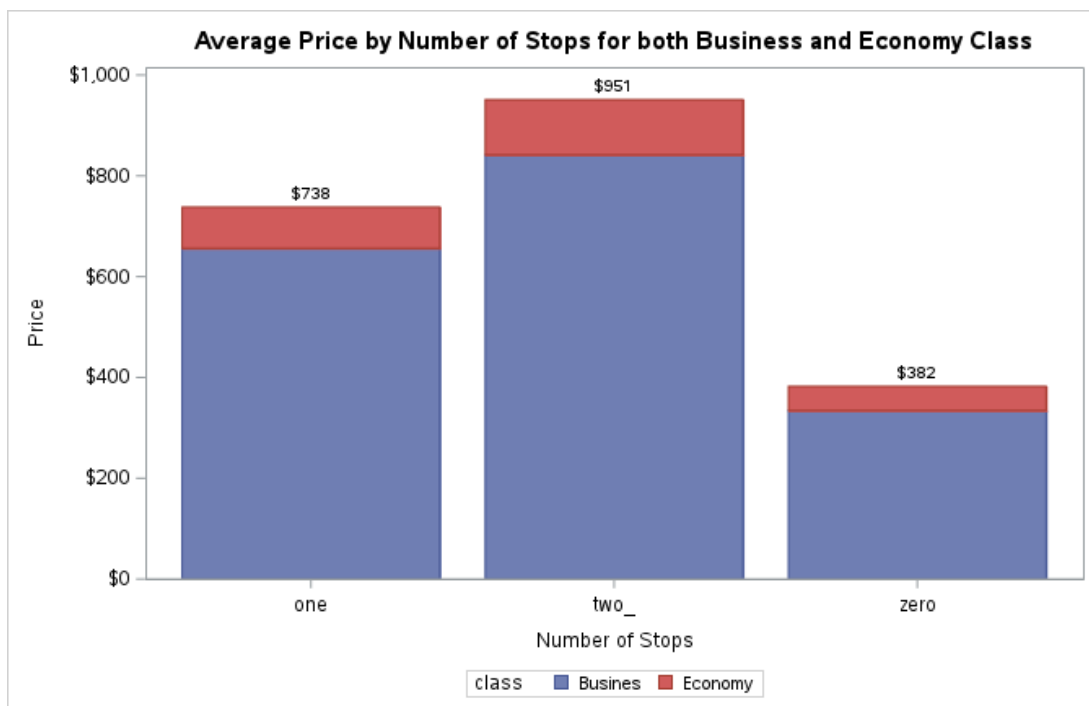


Figure 13 – Bar plot visualizing the average price by number of stops for both Business and Economy Class

Figure 13 allows us to observe how ticket prices vary depending on the number of stops in the flight. Typically, non-stop flights tend to have higher average prices compared to flights with layovers or multiple stops. This trend may be attributed to the convenience and time-saving aspect of non-stop flights, which passengers are often willing to pay a premium for. Conversely, flights with layovers or multiple stops are usually priced lower to accommodate for the inconvenience and longer travel duration associated with such itineraries.

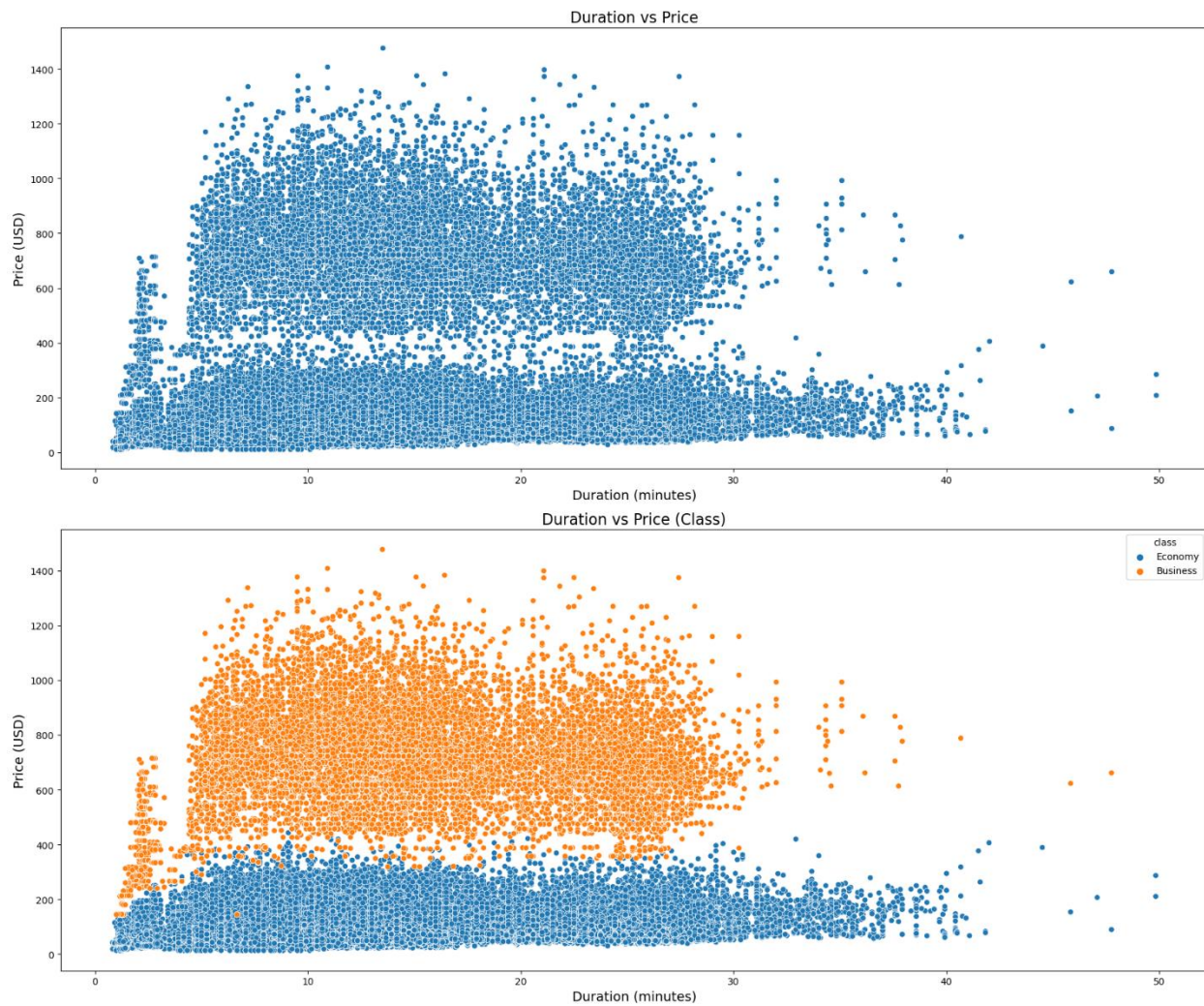


Figure 14 – Scatter Plot visualizing Duration vs. Price

There isn't a significant relationship between duration and price (solely) from the scatter plot. However, when we introduce Class into our analysis, it's obvious that tickets for Business class are more expensive than those for Economy class given the same duration, which implies the same flight. This is intuitively true as Business class is more expensive than Economy class on the same flight.

3.2 Machine Learning

3.2.1 Linear Regression Model

We employed linear regression modeling to predict flight prices based on several features such as airline, class, stops, and duration. This allowed us to build a predictive model that can estimate the price of a flight ticket given certain parameters.

Linear Regression Model to Predict Price				
Obs	Price_US_dollars	predicted	predicted_resid	r
1	71.436	49.4119	96.270	-24.834
2	71.436	49.5830	97.204	-25.768
3	71.472	51.5062	107.703	-36.231
4	71.460	50.5446	102.454	-30.994
5	71.460	50.6301	102.921	-31.461
6	71.460	50.6301	102.921	-31.461
7	72.720	50.3628	101.461	-28.741
8	72.720	50.4590	101.987	-29.267
9	71.448	48.3648	90.553	-19.105
10	71.448	48.4503	91.020	-19.572
11	71.448	48.4503	91.020	-19.572
12	71.448	48.5359	91.487	-20.039
13	71.460	46.2705	79.120	-7.660
14	71.460	46.2705	79.120	-7.660
15	71.460	46.3561	79.587	-8.127
16	71.460	46.4416	80.054	-8.594
17	71.460	47.2214	84.311	-12.851
18	71.460	47.3177	84.837	-13.377
19	71.388	83.9502	284.827	-213.439
20	71.388	88.3129	308.645	-237.257

Table 3 – Showing Linear Regression Model Price Prediction

Linear Regression Algorithm Train and Test Output.

Train Mean Squared Error: 7555.54

Test Mean Squared Error: 7580.14

Train Mean Absolute Error: 59.02

Test Mean Absolute Error: 58.85

Train R-squared: 0.90

Test R-squared: 0.90

The model demonstrates robust performance in predicting flight ticket prices, as evidenced by the low error metrics and high R-squared values. These results indicate that the Linear Regression model effectively captures the relationship between the input features and ticket prices, providing stakeholders with reliable insights for pricing strategies and decision-making.

3.2.2 Model evaluation.

Comparing the Predicted Prices with the Actual Prices

Training Data - Actual vs Predicted Price:

	Actual Price (USD)	Predicted Price (USD)	Price Difference (USD)
148417	162.288	79.830056	82.457944
36879	119.280	48.681584	70.598416
274531	671.796	617.304339	54.491661
166397	95.124	114.245494	-19.121494
272722	666.024	670.458941	-4.434941
183550	62.472	108.210190	-45.738190
47422	62.784	107.357235	-44.573235
255047	746.256	665.186128	81.069872
200793	87.744	77.180724	10.563276
37174	51.012	86.175525	-35.163525
190559	55.644	108.429891	-52.785891
43344	180.936	106.814446	74.121554
176994	81.036	90.789233	-9.753233
2038	150.840	136.642026	14.197974
117329	79.032	67.009883	12.022117
168414	80.808	84.779780	-3.971780
3054	69.672	80.838092	-11.166092
228452	493.212	600.387397	-107.175397
40589	47.952	88.954087	-41.002087
220730	566.604	655.700234	-89.096234

Testing Data - Actual vs Predicted Price:

	Actual Price (USD)	Predicted Price (USD)	Price Difference (USD)
27131	88.392	80.256533	8.135467
266857	777.972	648.915364	129.056636
141228	74.340	114.568583	-40.228583
288329	721.920	654.950667	66.969333
97334	78.936	65.394438	13.541562
135931	54.660	57.162131	-2.502131
290630	286.056	562.335055	-276.279055
141944	46.320	87.881431	-41.561431
244517	386.760	623.972897	-237.212897
215903	922.092	662.278326	259.813674
282552	457.188	664.216861	-207.028861
254237	726.096	612.561392	113.534608
183617	29.724	22.382141	7.341859
79761	86.640	98.259050	-11.619050
213925	394.308	563.304322	-168.996322
233936	248.208	515.745622	-267.537622
98502	86.544	119.195218	-32.651218
96776	93.144	57.278443	35.865557
228194	265.680	516.391800	-250.711800
27896	125.016	80.861249	44.154751

In both datasets, there are instances where the model's predictions closely match the actual prices, as indicated by minimal price differences. However, there are also instances where the model deviates significantly from the actual prices, leading to notable price differences.

These differences highlight areas where the model may need further refinement or where the input features may not fully capture the variability in ticket prices.

Overall, the comparison provides insights into the model's performance and areas for improvement, aiding stakeholders in decision-making related to pricing strategies and model enhancement.

4 Critical Comparison of SAS and Python

4.1 Introduction

The analysis done in this project has been carried out using two different analytics tools: SAS University Edition and Python. This section provides a critical comparison of the two analytics tools regarding the analysis done in Section 3 of this project. The comparison will cover a range of criteria including tool's efficiency, user-friendliness, scalability for large datasets, ease of use, and data visualization capabilities.

4.2 SAS University Edition Overview

SAS stands for Statistical Analysis Software. It was created in the year 1960 by the SAS Institute. From 1st January 1960, SAS was used for data management, business intelligence, Predictive Analysis, Descriptive and Prescriptive Analysis etc. Since then, many new statistical procedures and components were introduced in the software ([SAS - Overview \(tutorialspoint.com\)](https://www.tutorialspoint.com/sas/sas-overview.htm)). SAS University Edition is a free version of SAS software designed for educational purposes. It offers access to essential SAS products and tools, including SAS Studio, SAS/STAT, and SAS/IML, through a virtual machine that can be installed on personal computers. Users can learn data analysis, statistical modeling, and predictive analytics with extensive learning resources and community support. While it has limitations compared to the commercial version, it serves as a valuable platform for students, professors, and independent learners to develop analytical skills and explore data-driven solutions.

4.3 Python Overview

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently whereas other languages use punctuation, and it has fewer syntactical constructions than other languages.

Python is an open-source and cross-platform programming language. It is available for use under Python Software Foundation License (compatible to GNU General Public License) on all the major operating system platforms Linux, Windows, and Mac OS.

To facilitate new features and to maintain that readability, the Python Enhancement Proposal (PEP) process was developed. This process allows anyone to submit a PEP for a new feature, library, or other addition.

The design philosophy of Python emphasizes simplicity, readability, and unambiguity. Python is known for its batteries included approach as Python software is distributed with a comprehensive standard library of functions and modules ([Python - Overview \(tutorialspoint.com\)](https://www.tutorialspoint.com/python/python-overview.htm)).

4.4 Evaluation of SAS University Edition and Python

An in-depth evaluation of SAS University Edition and Python is presented in this section. The evaluation is done subject to a range of criteria under tool's efficiency, user-friendliness, scalability for large datasets, ease of use, and data visualization capabilities.

Tool's Efficiency

- SAS University Edition: SAS is known for its efficiency in handling large datasets and complex analytical tasks. It utilizes optimized algorithms and parallel processing capabilities to deliver fast performance.
- Python: Python's efficiency depends on the libraries and packages used for data analysis. Libraries like NumPy and pandas offer efficient data manipulation, while libraries like scikit-learn provide efficient machine learning algorithms.

User-friendliness

- SAS University Edition: SAS Studio, the interface used in SAS University Edition, provides a user-friendly environment with point-and-click features, drag-and-drop functionality, and intuitive menus. However, the learning curve for SAS programming language can be steep for beginners.
- Python: Python is generally considered user-friendly due to its simple syntax and readability. Jupyter Notebooks, commonly used for data analysis in Python, offer an interactive and user-friendly interface where users can mix code with explanatory text and visualizations.

Scalability for large datasets

- SAS University Edition: SAS is designed to handle large datasets efficiently, with features like in-memory processing and distributed computing options. SAS Viya, the cloud-native version of SAS, further enhances scalability by leveraging cloud resources.
- Python: Python's scalability for large datasets depends on the libraries and tools used. While libraries like pandas can handle moderately large datasets, Python may struggle with extremely large datasets compared to SAS.

Ease of Use

- SAS University Edition: SAS Studio offers a comprehensive environment with built-in tools for data manipulation, statistical analysis, and reporting. However, the proprietary nature of SAS and the complexity of its programming language may pose challenges for novice users.
- Python: Python's simplicity and versatility make it easy to learn and use for a wide range of tasks, including data analysis. The availability of extensive documentation, tutorials, and online resources further enhances its ease of use.

Data Visualization Capabilities

- SAS University Edition: SAS provides robust data visualization capabilities through tools like SAS Visual Analytics and SAS Visual Statistics. Users can create interactive and customizable visualizations to explore and communicate insights.
- Python: Python offers a variety of libraries for data visualization, including Matplotlib, Seaborn, and Plotly. These libraries provide a wide range of plotting options and customization features, allowing users to create publication-quality visualizations.

To quantify these observations, a score out of 10 has been allocated to each criteria for each tool in Table 4 below:

Criteria	SAS University Edition	Python
Tool's Efficiency	9	7
User-Friendliness	8	9
Scalability for Large Datasets	9	7
Ease of Use	7	9
Data Visualization Capabilities	9	8
Availability of Libraries	8	10
Community Support	8	9
Learning Resources	9	9
Integration with Other Tools	8	9
Flexibility and Customization	7	9
Deployment Options	8	9
Total	92	95

Table 4 – SAS University Edition and Python Evaluation Criteria Scoring

Based on the total scores, Python still has a slight edge over SAS University Edition. However, the choice between the two depends on specific project requirements, user expertise, and organizational preferences.

5 Conclusion

5.1 Analytics Insights

Airline Preferences: The analysis reveals insights into the popularity of airlines among travelers booking flights through the Ease My Trip website. Vistara emerges as the top performer, followed by Air India and Indigo. Understanding airline preferences can inform strategic decisions related to partnerships, marketing campaigns, and route optimization.

Price Dynamics: Ticket prices vary significantly based on factors such as airline, class, departure/arrival times, source/destination cities, and number of stops. For instance, business class tickets exhibit notably higher average prices compared to economy class, indicating premium pricing for enhanced services. Additionally, prices tend to increase significantly when tickets are purchased just one or two days before departure, highlighting the impact of booking timing on ticket prices.

Geographical Influence: The analysis demonstrates variations in ticket prices based on source and destination cities. Factors such as distance, popularity of routes, and demand-supply dynamics influence these price differentials. Understanding the pricing dynamics based on geographical factors can help airlines optimize pricing strategies and revenue management practices.

Class Preferences: While economy class accounts for most ticket sales, business class represents a smaller yet significant portion of the market. Understanding the price differentials between economy and business class is crucial for revenue management and pricing strategies, allowing stakeholders to tailor pricing structures to maximize revenue potential across different passenger segments.

Impact of Stops and Duration: Non-stop flights tend to have higher average prices compared to flights with layovers or multiple stops. This trend may be attributed to the convenience and time-saving aspect of non-stop flights, which passengers are often willing to pay a premium for. Understanding the impact of stops and duration on ticket prices can inform route planning and pricing strategies.

Overall, the analytics insights provide valuable information for stakeholders in the aviation industry, including airlines, travel agencies, and consumers. By leveraging these insights, stakeholders can make informed decisions related to pricing strategies, route planning, marketing efforts, and customer service enhancements to optimize competitiveness and profitability in the market.

5.2 Limitations and Future Work

The current analysis faces limitations stemming from the dataset's restricted scope and representativeness, as well as the simplicity of the utilized model, which overlooks external factors. To address these constraints, future research could focus on enriching the dataset with additional features such as passenger preferences and flight ratings, exploring more sophisticated modeling techniques like ensemble methods or neural networks, conducting a temporal analysis to understand pricing fluctuations over time, segmenting the dataset to uncover unique pricing patterns within different traveler demographics, and integrating external data sources such as

economic indicators and social media sentiment to provide a more holistic understanding of the factors influencing ticket prices in the aviation industry.

6 References

1. Nagesh, P., Naidu, K. B. J., Kowshik, P. N. V., & Sekhar, P. C. (2023). Airline Ticket Price Prediction Model. *International Journal for Research in Applied Science and Engineering Technology*, 11(3), 862-866. DOI: 10.22214/ijraset.2023.49537
2. Gupta, S., Gupta, N., Dheemansh Publication Hub, T., & Dheemansh Publication Hub, T. (2024). Flight Fare Prediction Using Machine Learning. *The Journal of Computational Science and Engineering*, ISSN: 2583-9055.
3. Laudon, K. C. & Laudon, J. P. (2006). *Management Information Systems: Managing the Digital Firm*. 10th ed. Prentice-Hall and Pearson Education.
4. Selçuk, A. M., & Avşar, Z. M. (2019). Dynamic pricing in airline revenue management. *Journal of Mathematical Analysis and Applications*, 478(2). DOI: 10.1016/j.jmaa.2019.06.012
5. [Python - Overview \(tutorialspoint.com\)](https://www.tutorialspoint.com/python/)
6. [SAS - Overview \(tutorialspoint.com\)](https://www.tutorialspoint.com/sas/)
7. [About Us and Company Profile - EaseMyTrip.com](https://www.easemytrip.com/about-us/)
8. <https://www.kaggle.com/datasets/shubhambathwal/flight-price-prediction>
9. [Untitled Diagram.drawio - Page-1 \(diagrams.net\)](https://drawio.net/)

7 Appendix 1 – SAS Code

```
/* Create a Project Library */

libname project '/home/u63777365/Business Analytics Assessment';
run;

/* Import the flight CSV file */
proc import datafile='/home/u63777365/Business Analytics Assessment/Clean_Dataset.csv'
            out=project.flight
            dbms=csv
            replace;
    getnames=yes;
run;

/* Print the content of the data */
proc contents data=project.flight;
run;

/* print the first 20 Observations of the data */
proc print data=project.flight (obs=20) noobs ;
run;

/* Data Preparation */

/*Dropping Rows with missing Values for all Variables*/
data project.flight;
    set project.flight;
    if not cmiss(of _all_);
run;

/* Summary statistics for numeric variables */
proc means data=project.flight;
    var duration days_left price;
run;

/* Convert duration to minutes */
data project.flight;
/* Rename duration to duration_hour to represent hour */
    set project.flight(rename=(duration=duration_hour));
    duration_minutes =duration_hour*60;
run;

/* Print 20 obs of the data */
proc print data=project.flight (obs=20);
run;
```

```

/* Convert price from india rupee to US dollars */
data project.flight;
/* convert flight ticket price to indian Rupee */
set project.flight(rename=(price=Price_Indian_Rupee));
/* 1 Rupee = 0.012 USD as of 29/04/2024 */
Price_US_dollars = Price_Indian_Rupee*0.012;
run;

/* Summary statistics for numeric variables */
proc means data=project.flight;
var duration_hour days_left Price_US_dollar;
run;

/* Print 20 obs of the data */
proc print data=project.flight (obs=20);
run;

proc print data=project.flight (obs=20);
format Price_US_dollars dollar10.2;
run;

/* Calculate frequency of airline */
proc freq data=project.flight noprint;
tables airline / out=airline_freq (rename=(count=Freq));
run;

/* Create a horizontal bar plot of tickets sold by different airlines */
proc sgplot data=airline_freq;
vbar airline /
response=Freq
datalabel
nostatlabel;
title 'Tickets Sold by Different Airlines';
xaxis label='Airlines';
yaxis label='Number of Tickets';
run;

/* Impact of buying tickets before departure */
/* Calculate average price and days_off by airline */
proc means data=project.flight noprint;
class airline;
var Price_US_dollars days_left;
output out=avg_airline(drop=_type_ _freq_) mean(Price_US_dollars)=avg_price mean(days_left)=avg_days_left;
run;

/* Check if prices varies for each price */
/* Create a bar plot of average price by airlines with different colors */
proc sgplot data=avg_airline;
vbar airline / response=avg_price
group=airline
datalabel
nostatlabel;
format avg_price dollar12.2;
title 'Average Price by Airlines in USD';
xaxis label='Airline';
yaxis label='Average Price';
run;

```

```

/* Plot bar plots for price vs. departure time and arrival time */
proc sgplot data=project.flight;
  vbar departure_time / response=Price_US_dollars stat=mean datalabel;
  xaxis label='Departure Time' fitpolicy=rotate;
  yaxis label='Price in USD';
  format Price_US_dollars dollar12.;
  title 'Average Price in USD by Departure Time';
run;

```

```

proc sgplot data=project.flight;
  vbar arrival_time / response=Price_US_dollars stat=mean datalabel;
  xaxis label='Departure Time' fitpolicy=rotate;
  yaxis label='Price in USD';
  format Price_US_dollars dollar12.;
  title 'Average Price in USD by Arrival Time';
run;

```

```

/* Plot KDE plots for price distribution by departure time */
proc sgpanel data=project.flight;
  panelby departure_time / layout=rowlattice columns=1 novarname;
  density Price_US_dollars / group=arrival_time;
  colaxis label='Price';
  rowaxis label='Density';
  format Price_US_dollars dollar12.;
  title 'Price Distribution by Departure Time';
run;

```

```

/* Plot KDE plots for price distribution by arrival time */
proc sgpanel data=project.flight;
  panelby arrival_time / layout=rowlattice columns=1 novarname;
  density Price_US_dollars / group=departure_time;
  colaxis label='Price';
  rowaxis label='Density';
  format Price_US_dollars dollar12.;
  title 'Price Distribution by Arrival Time';
run;

```

```

/* Create donut chart for share of class */
ods graphics / reset width=7in height=4.5in;
title 'Flight Class Distribution';
proc sgpie data=project.flight;
  styleattrs datacolors=(gold olive);
  donut class / holevalue holelabel='Count' ringsize=0.5;
run;

```

```

/* Create KDE plot for price distribution by class */
proc sgpanel data=project.flight;
  panelby class / novarname columns=1;
  density Price_US_dollars / group=class;
  colaxis label='Price';
  rowaxis label='Density';
  format Price_US_dollars dollar12.;
  title 'Price Distribution by Class';
run;

```

```

/* Create bar plot for number of stops vs. price */
proc sgplot data=project.flight;
  vbar stops / response=Price_US_dollars stat=mean group=class datalabel;
  xaxis label='Number of Stops';
  yaxis label='Price';
  format Price_US_dollars dollar12.;
  title 'Average Price by Number of Stops for both Business and Economy Class';
run;

/* Predicting Prices of Flight using Linear Regression Machine learning Model */
/* encode the Values */
data predictors;
  set project.flight;
  if class = 'Economy' then class_num = 0;
  else if class = 'Business' then class_num = 1;
  if airline = 'Indigo' then airline_num = 1;
  else if airline = 'Air_India' then airline_num = 2;
  else if airline = 'GO_FIRST' then airline_num = 3;
  else if airline = 'SpiceJet' then airline_num = 4;
  else if airline = 'Vistara' then airline_num = 5;
  else if airline = 'AirAsia' then airline_num = 6;
  if stops = 'zero' then stops_num = 0;
  else if stops = 'one' then stops_num = 1;
  else if stops = 'two' then stops_num = 2;
run;

/* Fit linear regression model with the right predictors */
proc reg data=predictors;
  model Price_US_dollars = airline_num class_num stops_num duration_minutes;
  title 'Linear Regression Model to Predict Price';
  output out=prediction_predicted predicted=predicted;
run;

/* Printing predicted data */
proc print data=prediction_predicted (obs=20);
run;

/* Compute model performance metrics */
proc reg data=prediction_predicted;
  model Price_US_dollars = predicted;
  output out=prediction_metrics p=predicted_resid r=r;
run;

/* Print model performance metrics */
proc print data=prediction_metrics(obs=20);
  var Price_US_dollars predicted predicted_resid r;
run;

```

8 Appendix 2 – Python Code

 jupyter Project Flight Pirce Prediction (1) Last Checkpoint: 2 minutes ago (autosaved)



```
File Edit View Insert Cell Kernel Widgets Help Not Trusted Python 3
```

```
In [13]: import numpy as np #
import pandas as pd #
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import warnings
warnings.filterwarnings("ignore")

In [14]: data = pd.read_csv('Clean_Dataset.csv', usecols=lambda col: col!='Unnamed: 0')

In [15]: data.head(20)
```

Out[15]:

	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class	duration	days_left	price
0	SpiceJet	SG-8709	Delhi	Evening	zero	Night	Mumbai	Economy	2.17	1	5953
1	SpiceJet	SG-8157	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.33	1	5953
2	AirAsia	I5-764	Delhi	Early_Morning	zero	Early_Morning	Mumbai	Economy	2.17	1	5956
3	Vistara	UK-995	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.25	1	5955
4	Vistara	UK-963	Delhi	Morning	zero	Morning	Mumbai	Economy	2.33	1	5955
5	Vistara	UK-945	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.33	1	5955
6	Vistara	UK-927	Delhi	Morning	zero	Morning	Mumbai	Economy	2.08	1	6060
7	Vistara	UK-951	Delhi	Afternoon	zero	Evening	Mumbai	Economy	2.17	1	6060
8	GO_FIRST	G8-334	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.17	1	5954

In [16]: `data.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300153 entries, 0 to 300152
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  -
0   airline                300153 non-null object
1   flight                 300153 non-null object
2   source_city            300153 non-null object
3   departure_time         300153 non-null object
4   stops                  300153 non-null object
5   arrival_time           300153 non-null object
6   destination_city       300153 non-null object
7   class                  300153 non-null object
8   duration                300153 non-null float64
9   days_left              300153 non-null int64
10  price                  300153 non-null int64
dtypes: float64(1), int64(2), object(8)
memory usage: 25.2+ MB
```

```
In [17]: data.describe()
```

```
Out[17]:
```

	duration	days_left	price
count	300153.000000	300153.000000	300153.000000
mean	12.221021	26.004751	20889.660523
std	7.191997	13.561004	22697.767366
min	0.830000	1.000000	1105.000000
25%	6.830000	15.000000	4783.000000
50%	11.250000	26.000000	7425.000000
75%	16.170000	38.000000	42521.000000
max	49.830000	49.000000	123071.000000

```
In [18]: data.isnull().sum()
```

```
Out[18]: airline      0
flight      0
source_city  0
departure_time  0
stops      0
arrival_time  0
destination_city  0
class      0
duration    0
days_left  0
price      0
dtype: int64
```

```
In [19]: #Convert the duration in hours to durations in minutes
data['duration_minutes'] = data['duration']*60
data.head(20)
```

Out[19]:

	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class	duration	days_left	price	duration_minutes
0	SpiceJet	SG-8709	Delhi	Evening	zero	Night	Mumbai	Economy	2.17	1	5953	130.2
1	SpiceJet	SG-8157	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.33	1	5953	139.8
2	AirAsia	I5-764	Delhi	Early_Morning	zero	Early_Morning	Mumbai	Economy	2.17	1	5956	130.2
3	Vistara	UK-995	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.25	1	5955	135.0
4	Vistara	UK-963	Delhi	Morning	zero	Morning	Mumbai	Economy	2.33	1	5955	139.8
5	Vistara	UK-945	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.33	1	5955	139.8
6	Vistara	UK-927	Delhi	Morning	zero	Morning	Mumbai	Economy	2.08	1	6060	124.8
7	Vistara	UK-951	Delhi	Afternoon	zero	Evening	Mumbai	Economy	2.17	1	6060	130.2
8	GO_FIRST	G8-334	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.17	1	5954	130.2
9	GO_FIRST	G8-336	Delhi	Afternoon	zero	Evening	Mumbai	Economy	2.25	1	5954	135.0
10	GO_FIRST	G8-392	Delhi	Afternoon	zero	Evening	Mumbai	Economy	2.25	1	5954	135.0
11	GO_FIRST	G8-338	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.33	1	5954	139.8
12	Indigo	6E-5001	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.17	1	5955	130.2
13	Indigo	6E-6202	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.17	1	5955	130.2
14	Indigo	6E-549	Delhi	Afternoon	zero	Evening	Mumbai	Economy	2.25	1	5955	135.0
15	Indigo	6E-6278	Delhi	Morning	zero	Morning	Mumbai	Economy	2.33	1	5955	139.8
16	Air_India	AI-887	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.08	1	5955	124.8
17	Air_India	AI-665	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.17	1	5955	130.2

```
In [20]: #Convert the price from indian rupee to dollars
data['price_usd'] = data['price']*0.012
data.head(20)
```

Out[20]:

	airline	flight	source_city	departure_time	stops	arrival_time	destination_city	class	duration	days_left	price	duration_minutes	price_usd
0	SpiceJet	SG-8709	Delhi	Evening	zero	Night	Mumbai	Economy	2.17	1	5953	130.2	71.436
1	SpiceJet	SG-8157	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.33	1	5953	139.8	71.436
2	AirAsia	I5-764	Delhi	Early_Morning	zero	Early_Morning	Mumbai	Economy	2.17	1	5956	130.2	71.472
3	Vistara	UK-995	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.25	1	5955	135.0	71.460
4	Vistara	UK-963	Delhi	Morning	zero	Morning	Mumbai	Economy	2.33	1	5955	139.8	71.460
5	Vistara	UK-945	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.33	1	5955	139.8	71.460
6	Vistara	UK-927	Delhi	Morning	zero	Morning	Mumbai	Economy	2.08	1	6060	124.8	72.720
7	Vistara	UK-951	Delhi	Afternoon	zero	Evening	Mumbai	Economy	2.17	1	6060	130.2	72.720
8	GO_FIRST	G8-334	Delhi	Early_Morning	zero	Morning	Mumbai	Economy	2.17	1	5954	130.2	71.448
9	GO_FIRST	G8-336	Delhi	Afternoon	zero	Evening	Mumbai	Economy	2.25	1	5954	135.0	71.448
10	GO_FIRST	G8-392	Delhi	Afternoon	zero	Evening	Mumbai	Economy	2.25	1	5954	135.0	71.448
11	GO_FIRST	G8-338	Delhi	Morning	zero	Afternoon	Mumbai	Economy	2.33	1	5954	139.8	71.448

```
In [21]: # Plotting the value counts of 'airline' column
plt.figure(figsize=(10, 6))
data['airline'].value_counts().plot(kind='barh', color='skyblue')
plt.title('Tickets Sold by Different Airlines')
plt.xlabel('Number of Tickets Sold')
plt.ylabel('Airline')
plt.gca().invert_yaxis() # Invert y-axis to have the highest count on top
plt.show()
```

```
In [22]: # Q1 Does price vary with Airlines?
# create a barplot
plt.figure(figsize=(10, 6))
sns.barplot(x='airline', y='price_usd', data=data, errorbar=None, palette='muted')
plt.title('Average Price by Airlines')
plt.xlabel('Airline')
plt.ylabel('Average Price (USD)')
plt.xticks(rotation=45) # Rotate x-axis labels for better readability
plt.show()
```

```
In [23]: # Q2 How is the price affected when tickets are bought in just 1 or 2 days before departure?

# Plotting the line plot
plt.figure(figsize=(10, 6))
sns.lineplot(x='days_left', y='price_usd', hue='airline', data=data, errorbar=None)
plt.title('Price vs. Number of Days before Departure')
plt.ylabel('Price (USD)')
plt.xlabel('Number of Days before Departure')
plt.xticks(range(data['days_left'].min(), data['days_left'].max(), 3))
plt.legend(title='Airline', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True) # Add a grid
plt.show()

# As we can see from the chart, when there are one or two days remaining for departure, the ticket
# price is higher than others, regardless of the Airline. But maybe because of different price strategy, ticket price at
# Vistara and Air India in the last day are significantly lower than two days before departure.
```

```
In [24]: # Q3 Does ticket price change based on the departure time and arrival time?
time_order = ['Early_Morning', 'Morning', 'Afternoon', 'Evening', 'Night', 'Late_Night']

# Plotting departure and arrival time vs price
plt.figure(figsize=(18, 10))
plt.subplot(1, 2, 1)
sns.barplot(x='departure_time', y='price_usd', hue='airline', order=time_order, data=data, palette='muted')
plt.xlabel('Departure Time', fontsize=16)
plt.ylabel('Price (USD)', fontsize=16)
plt.xticks(fontsize=12, rotation=45)
plt.title('Average Price by Departure Time', fontsize=16)
plt.legend(title='Airline', fontsize=12, title_fontsize='14')

plt.subplot(1, 2, 2)
sns.barplot(x='arrival_time', y='price_usd', hue='airline', order=time_order, data=data, palette='muted')
plt.xlabel('Arrival Time', fontsize=16)
plt.ylabel('Price (USD)', fontsize=16)
plt.xticks(fontsize=12, rotation=45)
plt.title('Average Price by Arrival Time', fontsize=16)
plt.legend(title='Airline', fontsize=12, title_fontsize='14')

plt.tight_layout()
plt.show();

# Plotting price distribution by departure and arrival time
plt.figure(figsize=(18, 10))
plt.subplot(2, 1, 1)
sns.kdeplot(x='price_usd', hue='departure_time', data=data, hue_order=time_order, fill=True, palette='muted')
plt.title('Price Distribution by Departure Time', fontsize=16)
plt.xlabel('Price (USD)', fontsize=16)
plt.ylabel('Density', fontsize=16)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.legend(title='Departure Time', fontsize=12, title_fontsize='14')

plt.subplot(2, 1, 2)
sns.kdeplot(x='price_usd', hue='arrival_time', data=data, hue_order=time_order, fill=True, palette='muted')
plt.title('Price Distribution by Arrival Time', fontsize=16)
plt.xlabel('Price (USD)', fontsize=16)
plt.ylabel('Density', fontsize=16)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
plt.legend(title='Arrival Time', fontsize=12, title_fontsize='14')
plt.tight_layout()
plt.show()
```

```
In [25]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 10))
sns.boxplot(x='departure_time', y='price_usd', data=data, ax=ax1)
ax1.set_xlabel('Departure Time', fontsize=20)
ax1.set_ylabel('Price', fontsize=20)
ax1.set_xticklabels(time_order, fontsize=15, rotation=45)

sns.boxplot(x='arrival_time', y='price_usd', data=data, ax=ax2)
ax2.set_xlabel('Arrival Time', fontsize=20)
ax2.set_ylabel('Price', fontsize=20)
ax2.set_xticklabels(time_order, fontsize=15, rotation=45);
```

```
In [26]: # Q4 How the price changes with change in Source and Destination?

# Create subplots for bar plots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 10))

# Plot bar plot for source city vs average price
sns.barplot(x='source_city', y='price_usd', data=data, ax=ax1, errorbar=None, palette='muted')
ax1.set_xlabel('Source City', fontsize=16)
ax1.set_ylabel('Average Price (USD)', fontsize=16)
ax1.set_title('Average Price by Source City', fontsize=16)

# Plot bar plot for destination city vs average price
sns.barplot(x='destination_city', y='price_usd', data=data, ax=ax2, errorbar=None, palette='muted')
ax2.set_xlabel('Destination City', fontsize=16)
ax2.set_ylabel('Average Price (USD)', fontsize=16)
ax2.set_title('Average Price by Destination City', fontsize=16)
plt.show()

# Create subplots for KDE plots
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(18, 10))

# Plot KDE plot for price distribution by source city
sns.kdeplot(x='price_usd', hue='source_city', data=data, ax=ax1)
ax1.set_title('Price Distribution by Source City', fontsize=16)
ax1.set_xlabel('Price (USD)', fontsize=16)
ax1.set_ylabel('Density', fontsize=16)

# Plot KDE plot for price distribution by destination city
sns.kdeplot(x='price_usd', hue='destination_city', data=data, ax=ax2)
ax2.set_title('Price Distribution by Destination City', fontsize=16)
ax2.set_xlabel('Price (USD)', fontsize=16)
ax2.set_ylabel('Density', fontsize=16)

# Integrate insightful analysis into the code comments
"""
These KDE plots show the distribution of ticket prices based on the source and destination cities:

- The distributions indicate that ticket prices vary significantly based on both source and destination cities.
- The differences in price distribution could be attributed to factors such as distance, popularity of routes,
and demand-supply dynamics.
"""

plt.tight_layout()
plt.show()
```

```
In [27]: # Q5 How does the ticket price vary between Economy and Business class?
# Create subplots for bar plots
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 10))

# Plot bar plot for class vs average price
sns.barplot(x='class', y='price_usd', data=data, ax=ax1, palette='muted')
ax1.set_xlabel('Class', fontsize=16)
ax1.set_ylabel('Average Price (USD)', fontsize=16)
ax1.set_title('Average Price by Class', fontsize=16)

# Plot bar plot for class vs count
data['class'].value_counts().plot(kind='pie', autopct='%1.1f%%', ax=ax2)
ax2.set_ylabel('')
ax2.set_title('Ticket Distribution by Class', fontsize=16)

plt.show()
```

```
In [28]: # Create a bar plot to show the distribution of prices for each number of stops
plt.figure(figsize=(10, 6))
sns.barplot(x='stops', y='price_usd', data=data, palette='muted')
plt.title('Price Distribution by Number of Stops')
plt.xlabel('Number of Stops')
plt.ylabel('Price (USD)')
plt.show()
```

```
In [29]: # Create subplots
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(18, 15))

# Scatter plot for Duration vs Price
sns.scatterplot(x='duration', y='price_usd', data=data, ax=ax1)
ax1.set_title('Duration vs Price', fontsize=16)
ax1.set_xlabel('Duration (minutes)', fontsize=14)
ax1.set_ylabel('Price (USD)', fontsize=14)

# Scatter plot for Duration vs Price with Class
sns.scatterplot(x='duration', y='price_usd', hue='class', data=data, ax=ax2)
ax2.set_title('Duration vs Price (Class)', fontsize=16)
ax2.set_xlabel('Duration (minutes)', fontsize=14)
ax2.set_ylabel('Price (USD)', fontsize=14)

plt.tight_layout()
plt.show()
```

MACHINE LEARNING ALGORITHM

linear Regression Algorithm will be made use of in this project for predicting flight prices

```
In [30]: # Encode categorical variables
label_encoder = LabelEncoder()
data['airline_num'] = label_encoder.fit_transform(data['airline'])
data['class_num'] = label_encoder.fit_transform(data['class'])
data['stops_num'] = label_encoder.fit_transform(data['stops'])

In [31]: # Define predictors and target variable
X = data[['airline_num', 'class_num', 'stops_num', 'duration_minutes']]
y = data['price_usd']

In [32]: # Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

In [33]: # Train the model
model = LinearRegression()
model.fit(X_train, y_train)

In [34]: # Make predictions
y_pred_train = model.predict(X_train)
y_pred_test = model.predict(X_test)

In [35]: # Calculate Mean Squared Error
train_mse = mean_squared_error(y_train, y_pred_train)
test_mse = mean_squared_error(y_test, y_pred_test)

# Calculate Mean Absolute Error
train_mae = mean_absolute_error(y_train, y_pred_train)
test_mae = mean_absolute_error(y_test, y_pred_test)

# Calculate R-squared
train_r2 = r2_score(y_train, y_pred_train)
test_r2 = r2_score(y_test, y_pred_test)

In [36]: print(f"Train Mean Squared Error: {train_mse:.2f}")
print(f"Test Mean Squared Error: {test_mse:.2f}")
print(f"Train Mean Absolute Error: {train_mae:.2f}")
print(f"Test Mean Absolute Error: {test_mae:.2f}")
print(f"Train R-squared: {train_r2:.2f}")
print(f"Test R-squared: {test_r2:.2f}")

In [38]: # Create a DataFrame for actual and predicted prices
results_train = pd.DataFrame({'Actual Price (USD)': y_train, 'Predicted Price (USD)': y_pred_train})
results_test = pd.DataFrame({'Actual Price (USD)': y_test, 'Predicted Price (USD)': y_pred_test})

# Calculate the differences between actual and predicted prices
results_train['Price Difference (USD)'] = results_train['Actual Price (USD)'] - results_train['Predicted Price (USD)']
results_test['Price Difference (USD)'] = results_test['Actual Price (USD)'] - results_test['Predicted Price (USD)']

# Display the DataFrame
print("Training Data - Actual vs Predicted Price:")
print(results_train.head(20)) # Displaying the first 10 rows
print("\n")
print("Testing Data - Actual vs Predicted Price:")
print(results_test.head(20)) # Displaying the first 10 rows
```